

DOCUMENTAÇÃO DO PROJETO

Sistema de Gerenciamento de Academia

Avaliação 2 - Desenvolvimento Orientado a Objetos

Java + Swing + MySQL (JDBC)

Sumário

Sumário	2
1. Introdução	3
1.1 Objetivo do Sistema	3
1.2 Tecnologias Utilizadas	3
2. Arquitetura do Sistema	3
2.1 Camadas	3
2.2 Padrão de Acesso a Dados	3
3. Modelo de Dados (Classes)	4
3.1 Hierarquia de Pessoas	4
3.2 Hierarquia de Exercícios	4
3.3 Plano de Treino	4
3.4 Hierarquia de Formas de Pagamento	4
3.5 Pagamento e Compra	4
3.6 Login	5
4. Interfaces Gráficas (Telas)	5
4.1 GuiLogin	5
4.2 GuiMenuPrincipal	5
4.3 GuiCadastroAluno	5
4.4 GuiPagamento	6
4.5 GuiCadastroProfessor	6
4.6 GuiCadastroPlanoTreino	6
4.7 GuiPlanoTreinoConsulta	7
5. Banco de Dados (MySQL)	7
5.1 Tabelas	7
5.2 Padrão de Operações	8
6. Divisão de Trabalho da Equipe	8
7. Considerações Finais	8

1. Introdução

Este documento descreve a estrutura, as funcionalidades e a arquitetura do Sistema de Gerenciamento de Academia, desenvolvido em Java com interface gráfica Swing e persistência de dados em MySQL via JDBC. O sistema foi desenvolvido em equipe como parte da Avaliação 2 da disciplina, seguindo o conteúdo dos capítulos 8, 9 e 12 do livro “Java 8 - Ensino Didático”, de Sérgio Furgeri.

1.1 Objetivo do Sistema

O sistema tem como objetivo gerenciar o cadastro de alunos e professores de uma academia, controlar os planos de treino atribuídos a cada aluno (com seus respectivos exercícios) e registrar os pagamentos realizados, suportando múltiplas formas de pagamento (Dinheiro, Pix, Cartão de Crédito e Cartão de Débito).

1.2 Tecnologias Utilizadas

- Linguagem: Java (JDK 21)
- Interface gráfica: Java Swing
- Banco de dados: MySQL
- Conectividade: JDBC (com.mysql.cj.jdbc.Driver)
- IDE: Eclipse IDE
- Controle de versão: Git e GitHub
- Diagramação UML: draw.io

2. Arquitetura do Sistema

O sistema foi organizado em camadas, separando responsabilidades de interface, regras de negócio e acesso ao banco de dados:

2.1 Camadas

Camada	Responsabilidade
views (GUI)	Telas Swing responsáveis pela interação com o usuário (GuiLogin, GuiMenuPrincipal, GuiCadastroAluno, GuiPagamento, etc.)
models	Classes de domínio que representam as entidades do sistema (Aluno, Professor, PlanoTreino, Pagamento, Exercícios, etc.)
dao (models)	Classes de acesso a dados (AlunoDAO, PagamentoDAO, LoginDAO, ProfessorDAO, PlanoTreinoDAO), responsáveis pela comunicação com o banco MySQL
Bd / MySQL	Classe de conexão e o banco de dados relacional, onde os dados são persistidos

2.2 Padrão de Acesso a Dados

Todas as classes DAO implementam a interface OperacaoBD, que define dois métodos principais:

- `localizar()`: boolean — busca um registro no banco e preenche o objeto correspondente
- `atualizar(TipoOperacaoBd operacao)`: String — realiza inclusão, alteração ou exclusão de acordo com o enum `TipoOperacaoBd` (INCLUSAO, ALTERACAO, EXCLUSAO)

3. Modelo de Dados (Classes)

3.1 Hierarquia de Pessoas

A classe abstrata `Pessoa` concentra os atributos comuns (nome, cpf, dataNascimento, numeroTelefone) e é especializada em:

- `Aluno` — adiciona o atributo `matricula` (int), utilizado como identificador do aluno
- `Professor` — adiciona os atributos `cref` e `especialidade`

3.2 Hierarquia de Exercícios

A classe abstrata `Exercicios` define os atributos `nomeExercicio` e `descricao`, além do método abstrato `getValorUnitario()`. Essa hierarquia se divide em duas subcategorias:

- `ExercicioComRepeticao` — exercícios cujo valor é calculado por repetição (`valorPorRepeticao`). Inclui 23 subclasses, como `AgachamentoLivre`, `SupinoRetoComBarra`, `BicepsRoscaComHalter`, `LegPress45`, entre outros.
- `ExercicioComTempo` — exercícios cujo valor é calculado por segundo de execução (`valorPorSegundo`). Inclui 7 subclasses, como `Esteira`, `Bicicleta`, `Elptico`, `PranchaFrontal`, `PularCorda`, `Escada` e `WallSit`.

Cada subclasse define seu próprio valor unitário no construtor (ex.: `AgachamentoLivre` custa 0.14 por repetição), permitindo o cálculo automático do valor total de um plano de treino.

3.3 Plano de Treino

A classe `PlanoTreino` relaciona um `Aluno`, um `Professor` responsável, uma data de criação, um array de `Exercicios` e um array paralelo com a quantidade de cada exercício. O método `calcularValorTotal()` percorre os exercícios e multiplica o valor unitário de cada um pela quantidade correspondente, somando o resultado.

3.4 Hierarquia de Formas de Pagamento

A classe abstrata `FormaPagamento` define os atributos `descricao` e `id`, além do método abstrato `pagar()`: boolean, que valida se os dados daquela forma de pagamento estão completos.

- `Dinheiro` — `pagar()` sempre retorna true; possui também o método `darTroco(valorPago, valorCompra)`
- `Pix` — exige chave e tipo (enum `TipoChave`: CPF, TELEFONE, EMAIL, CHAVE_ALEATORIA)
- `Cartao` (abstrata) — define o atributo `bandeira` (enum `BandeiraCartao`: VISA, MASTERCARD, AMERICAN_EXPRESS, HIPERCARD, ELO)
 - `CartaoCredito` — adiciona `quantidadeParcela` (byte); `pagar()` exige bandeira e ao menos 1 parcela
 - `CartaoDebito` — `pagar()` exige apenas a bandeira

3.5 Pagamento e Compra

A classe Pagamento armazena um array de FormaPagamento, o valor total e um id. O método efetuarPagamento() percorre todas as formas de pagamento associadas e retorna true apenas se todas forem válidas (pagar() == true para todas) e o valor for maior que zero.

A classe Compra relaciona um array de PlanoTreino a um Pagamento, e o método SomaTotal() soma o valor de cada plano de treino chamando calcularValorTotal().

3.6 Login

A classe Login representa a autenticação do sistema, com os atributos id, codigo e senhaHash. A senha é armazenada como hash (gerado a partir de String.hashCode()), nunca em texto puro. O método validarLogin(codigo, senha) compara o código e o hash da senha informada com os valores armazenados.

4. Interfaces Gráficas (Telas)

Todas as telas foram desenvolvidas utilizando Java Swing, com layout absoluto (setBounds) para posicionamento dos componentes.

4.1 GuiLogin

Tela inicial do sistema. Contém campos para código e senha, e os botões Logar e Cancelar.

- Ao clicar em Logar, o sistema cria um objeto Login, conecta ao banco via classe Bd e utiliza o LoginDAO para localizar o registro pelo código informado
- Se encontrado, valida a senha com login.validarLogin(); em caso de sucesso, abre o GuiMenuPrincipal
- Em caso de falha, exibe uma mensagem de erro ao usuário

4.2 GuiMenuPrincipal

Tela principal do sistema, organizada com uma barra de menus (JMenuBar) dividida em quatro categorias:

- Arquivo — opção Sair, que encerra a aplicação
- Aluno — opções Pagamento e Consulta Plano de Treino
- Professor — opções Cadastro Plano de Treino, Cadastro e Alteração de Aluno e Cadastro e Alteração de Professor
- Ajuda — opção Sobre..., exibindo os créditos da equipe

4.3 GuiCadastroAluno

Tela responsável pelo CRUD (Create, Read, Update, Delete) de alunos, contendo campos para matrícula, nome, CPF, data de nascimento e telefone. Possui cinco botões:

- Salvar — insere um novo aluno no banco (AlunoDAO.atualizar com TipoOperacaoBd.INCLUSAO)
- Buscar — localiza um aluno pela matrícula e preenche os demais campos (AlunoDAO.localizar)
- Alterar — atualiza os dados de um aluno existente (ALTERACAO)
- Excluir — remove um aluno do banco, após confirmação do usuário (EXCLUSAO)
- Limpar — limpa todos os campos do formulário

4.4 GuiPagamento

Tela responsável pelo registro de pagamentos. Contém um campo para o valor e um combo box para a forma de pagamento (dinheiro, pix, cartão de crédito, cartão de débito). De acordo com a opção selecionada, campos adicionais são exibidos ou ocultados dinamicamente:

- Pix — exibe campos de chave Pix e tipo de chave
- Cartão de Crédito — exibe campos de bandeira e número de parcelas
- Cartão de Débito — exibe apenas o campo de bandeira
- Dinheiro — não exibe campos adicionais

Ao clicar em Pagar, o sistema monta o objeto FormaPagamento correspondente, cria um Pagamento, válida com efetuarPagamento() e persiste no banco via PagamentoDAO (INCLUSAO).

4.5 GuiCadastroProfessor

Tela responsável pelo CRUD (Create, Read, Update, Delete) de professores, contendo campos para nome, CPF, data de nascimento, telefone, CREF e especialidade. Possui cinco botões:

- Salvar — insere um novo professor no banco de dados utilizando ProfessorDAO.atualizar com TipoOperacaoBd.INCLUSAO
- Buscar — localiza um professor pelo CPF informado e preenche automaticamente os demais campos da tela através do método ProfessorDAO.localizar
- Alterar — atualiza os dados de um professor já cadastrado utilizando TipoOperacaoBd.ALTERACAO
- Excluir — remove um professor do banco de dados após confirmação do usuário utilizando TipoOperacaoBd.EXCLUSAO
- Limpar — remove os dados preenchidos nos campos do formulário

Durante o preenchimento dos dados, o sistema realiza validações dos campos obrigatórios e da data de nascimento antes de persistir as informações.

4.6 GuiCadastroPlanoTreino

Tela responsável pelo cadastro e manutenção de planos de treino. Contém campos para matrícula do aluno, CPF do professor responsável, data de criação do treino e uma lista de exercícios organizados por grupos musculares, acompanhados de controles para definição das quantidades.

Os exercícios disponíveis são distribuídos entre os grupos:

- Peito, Bíceps, Tríceps, Ombro, Costas, Abdômen, Quadríceps, Posterior de Coxa, Glúteos, Cárdio.

A tela possui os seguintes botões:

- Salvar — cria um novo plano de treino associando aluno, professor e exercícios selecionados, persistindo os dados através de PlanoTreinoDAO.atualizar com TipoOperacaoBd.INCLUSAO
- Alterar — atualiza um plano de treino já existente utilizando TipoOperacaoBd.ALTERACAO

- Excluir — remove um plano de treino do banco de dados após confirmação do usuário utilizando TipoOperacaoBd.EXCLUSAO
- Limpar — reinicia todos os campos da tela, desmarca os exercícios selecionados e redefine as quantidades para zero

Durante o processo de cadastro, os exercícios selecionados são armazenados temporariamente em listas, convertidos para vetores e associados ao objeto PlanoTreino, juntamente com suas respectivas quantidades.

4.7 GuiPlanoTreinoConsulta

Tela responsável pela consulta de planos de treino cadastrados no sistema. Possui um campo para informar o ID do plano de treino e um botão para realizar a busca.

- Consultar Plano — localiza um plano de treino pelo ID informado utilizando PlanoTreinoDAO.localizar

Quando o plano é encontrado, o sistema:

- Recupera os exercícios associados ao plano
- Exibe os exercícios em uma tabela contendo nome do exercício, quantidade, valor unitário e subtotal
- Calcula automaticamente o valor total do plano através do método calcularValorTotal()

Caso o plano não seja encontrado, uma mensagem é exibida ao usuário e a tabela é limpa automaticamente.

A interface utiliza um componente JTable dentro de um JScrollPane para exibir os exercícios recuperados do banco de dados de forma organizada.

5. Banco de Dados (MySQL)

O banco de dados utilizado é o MySQL, acessado por meio da classe Bd, que encapsula a conexão JDBC (driver com.mysql.cj.jdbc.Driver).

5.1 Tabelas

Tabela	Descrição
aluno	Armazena matrícula (chave primária), nome, CPF, data de nascimento e telefone dos alunos
professor	Armazena CPF (chave primária), nome, data de nascimento, telefone, CREF e especialidade
planoTreino	Armazena id, matrícula do aluno, CPF do professor responsável e data de criação do plano
exercicios PlanoTreino	Relaciona cada plano de treino aos exercícios escolhidos e suas respectivas quantidades
pagamento	Armazena id (auto incremento) e valor do pagamento

formaPagamento	Armazena o tipo de pagamento (DINHEIRO, PIX, CARTAO_CREDITO, CARTAO_DEBITO) e seus detalhes (chave Pix, tipo de chave, bandeira, parcelas), relacionado ao id do pagamento
login	Armazena id, código e hash da senha para autenticação no sistema

5.2 Padrão de Operações

As operações de inclusão, alteração e exclusão são controladas pelo enum TipoOperacaoBd, utilizado em todas as classes DAO:

```
public enum TipoOperacaoBd {
    INCLUSAO,
    ALTERACAO,
    EXCLUSAO
}
```

6. Divisão de Trabalho da Equipe

Conforme solicitado nos requisitos da avaliação, cada membro da equipe foi responsável por um conjunto de classes e funcionalidades, identificadas nos commits e branches individuais do repositório no GitHub.

- Membro 1 — responsável pelas classes destacadas em laranja na UML: Pessoa, Aluno, Professor, Login, FormaPagamento e suas subclasses (Dinheiro, Pix, Cartao, CartaoCredito, CartaoDebito), Pagamento, Compra, além dos DAOs AlunoDAO, LoginDAO e PagamentoDAO e das telas GuiLogin, GuiCadastroAluno e GuiPagamento.
- Demais membros — responsáveis pelas classes destacadas em azul na UML: hierarquia de Exercícios e suas subclasses, PlanoTreino, ProfessorDAO, PlanoTreinoDAO e telas relacionadas (GuiMenuPrincipal, GuiCadastroProfessor, GuiCadastroPlanoTreino, GuiPlanoTreinoConsulta).

O versionamento foi feito utilizando o GitHub, com um branch individual por integrante e integração na branch main por meio de Pull Requests.

7. Considerações Finais

O Sistema de Gerenciamento de Academia atende aos requisitos propostos, implementando conceitos de orientação a objetos como herança, polimorfismo, abstração e interfaces, além de persistência de dados em MySQL e interface gráfica funcional desenvolvida em Java Swing. O sistema permite o cadastro e gerenciamento de alunos, professores, planos de treino e pagamentos, com cálculo automático de valores baseado nos exercícios selecionados.

A documentação completa do projeto também inclui o diagrama UML de classes (arquivo .drawio), os scripts SQL de criação e população do banco de dados, e a apresentação em PDF com a demonstração das telas em funcionamento.